Compiled AI: Deterministic Code Generation for LLM-Based Workflow Automation

Geert Trooskens[1] Aaron Karlsberg[1] Anmol Sharma[1] Lamara De Brouwer[1]
Max Van Puyvelde[2] Walter A. De Brouwer[2]

[1]XY.AI Labs, Palo Alto, CA
[2]Stanford University School of Medicine, Stanford, CA

foundation@xy.ai

January 2026

Abstract

We study *compiled AI*, a paradigm in which large language models generate executable code artifacts during a compilation phase, after which workflows execute deterministically without further model invocation. By constraining generation to narrow business-logic functions embedded in validated templates, compiled AI trades runtime flexibility for predictability, auditability, cost efficiency, and reduced security exposure, while retaining the ability to improve artifacts through observed execution patterns.

We introduce (i) a system architecture for constrained LLM-based code generation, (ii) a four-stage generation-and-validation pipeline that converts probabilistic model output into production-ready code artifacts, and (iii) an evaluation framework that measures operational metrics including token amortization, determinism, reliability, security, and cost. In preliminary evaluation on function-calling tasks (n=400 across BFCL categories), compiled AI achieves 96% task completion with zero execution tokens, breaking even with runtime inference at approximately 17 transactions and reducing token consumption by 57× at 1,000 transactions. Agent frameworks like LangChain and AutoGen consume 34–46% more tokens than direct LLM, further favoring compiled approaches. Security evaluation across 135 test cases demonstrates 96.7% accuracy on prompt injection detection and 87.5% on static code safety analysis with zero false positives.

1 Introduction

Large language models (LLMs) are increasingly deployed to automate enterprise workflows, evolving from question-answering systems toward autonomous agent architectures. Recent frameworks enable LLMs to reason over tools, APIs, and multi-step tasks at runtime, often through conversational or multi-agent interaction patterns [Wu et al., 2024, CrewAI, 2024, LangChain, 2022]. While these approaches demonstrate flexibility, they rely on repeated model invocation during execution, leading to high token consumption, variable latency, and non-deterministic behavior.

Empirical studies and industrial deployments have highlighted persistent reliability challenges in runtime agent systems, particularly in multi-step business workflows where success rates degrade due to specification ambiguity, inter-agent coordination failures, and stochastic model outputs [Cemri et al., 2025, Atil et al., 2024]. These limitations pose obstacles to deployment in enterprise environments, where determinism, auditability, and cost predictability are often strict requirements.

This paper investigates an alternative design point: *compiled AI*, a paradigm in which LLMs generate executable code artifacts during a one-time compilation phase, after which workflows execute deterministically without further model inference. The key observation motivating this work is that many enterprise workflows require intelligence to *design* but not to *execute* repeatedly. Once business logic is specified and validated, repeated execution can be handled by conventional software infrastructure, with full observability that enables targeted refinement of the compiled artifacts over time.

Compiled AI represents a trade-off between flexibility and determinism. Rather than interpreting natural language or reasoning at runtime, an LLM is constrained to generate narrow business-logic functions within pre-validated templates. These generated artifacts are subjected to a multi-stage validation pipeline (security analysis, syntactic verification, execution testing, and output accuracy checks) before deployment. After validation, workflows execute as static code with predictable latency, zero stochasticity, and near-zero marginal inference cost.

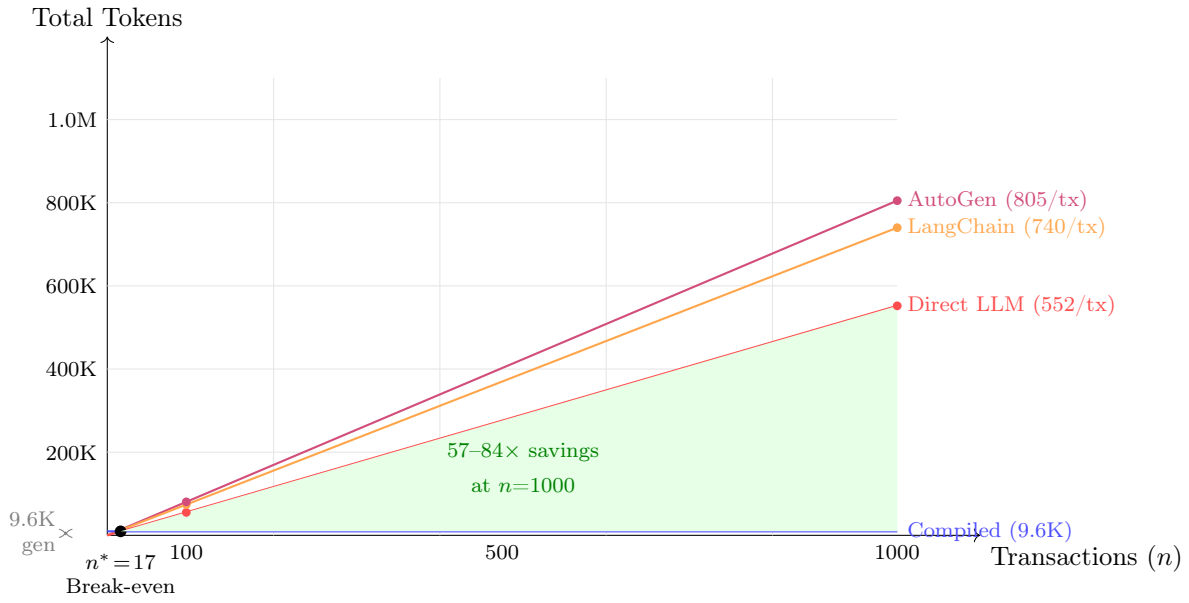

Figure 1: Token consumption comparison across baselines (BFCL benchmark, n=400). Direct LLM consumes 552 tokens/tx; agent frameworks add overhead—LangChain (740/tx, +34%) and AutoGen (805/tx, +46%). Compiled AI incurs a one-time 9,600 token generation cost, then executes at zero marginal cost. Break-even vs Direct LLM at $n^* \approx 17$ transactions; at 1,000 transactions, compiled AI reduces consumption by 57× vs Direct LLM and 84× vs AutoGen.

Contributions. This paper makes three contributions:

- We introduce an architecture for constrained LLM-based code generation in which models produce bounded business-logic functions embedded in durable workflow templates, enabling deterministic execution (Section 3).
- We present a four-stage generation-and-validation pipeline that empirically converts probabilistic model output into production-ready code artifacts through iterative regeneration (Section 3).

- We define an evaluation framework that measures the operational viability of LLM-based workflow systems using metrics for token amortization, determinism, reliability, validation effectiveness, and cost (Section 4).

Our goal is not to replace runtime inference in all settings, but to provide a principled basis for identifying regimes (such as well-specified, high-volume workflows) where compilation yields operational advantages.

2 Related Work

Novelty and Scope. Prior work on LLM-based agents, program-aided reasoning, and workflow synthesis has focused primarily on model capability: whether systems can generate or reason over programs. In contrast, this work treats production viability as a first-class concern. Our contributions are not a new code generation algorithm per se, but a systems-oriented study of how probabilistic LLM outputs can be converted into deterministic, auditable execution artifacts. To our knowledge, this is the first systematic evaluation of LLM-based workflow automation using operational and economic metrics such as token amortization, execution determinism, validation convergence, and cost at scale.

2.1 The Reliability Crisis in LLM Agents

Recent empirical studies reveal a fundamental reliability problem in multi-agent LLM systems. Cemri et al. [2025] analyzed 1,600+ annotated traces across seven major frameworks (MetaGPT, ChatDev, AutoGen), finding that **79% of failures stem from specification and coordination issues**, not infrastructure limitations. Their Multi-Agent System Failure Taxonomy (MAST) identifies specification failures (41.8%) and inter-agent coordination failures (36.9%) as dominant failure modes, with infrastructure problems accounting for only 16% of breakdowns.

Production deployments confirm this pattern. Salesforce's CRMArena-Pro benchmark shows success rates degrading from 58% (single-turn) to **35% in multi-turn interactions** [Salesforce, 2025]. Pan et al. [2025] surveyed 306 practitioners across 26 domains and found that 68% limit agents to $\leq$10 autonomous steps before requiring human intervention, with 74% depending primarily on human evaluation rather than automated metrics. Reliability was cited as the primary deployment challenge.

Critically, non-determinism persists even with temperature=0. Ouyang et al. [2023] demonstrated 18-75% output variance in code generation tasks due to architectural factors including Mixture-of-Experts routing and floating-point variance, fundamentally undermining reproducibility in interpreted AI approaches. Atil et al. [2024] found accuracy varies up to 15% across runs at temperature=0, with the gap between best and worst possible performance reaching 70%.

These findings motivate our core architectural choice: removing the LLM from the execution loop entirely, confining model invocation to a one-time compilation phase where variance can be detected and corrected before deployment.

2.2 Benchmark Saturation and Contamination

The apparent progress on code generation benchmarks masks significant methodological concerns. HumanEval is now saturated (>95% for frontier models), with Stanford's AI Index 2025 formally retiring it for frontier evaluation. More troubling, contamination studies reveal 8-18% of HumanEval appearing in pre-training data [Yang et al., 2023], with average 39.4% performance drops after decontamination [Xu et al., 2024].

Table 1: Code generation benchmark performance, December 2025. HumanEval is saturated; contamination-resistant benchmarks reveal substantial capability gaps.

Benchmark	Model	Score	Clean Score	Date
SWE-Bench Verified	Claude Opus 4.5	80.9%	—	Nov 2025
	GPT-5.2	80.0%	—	Dec 2025
SWE-Bench Pro	Claude Opus 4.1	—	22.7%	Jan 2026
	GPT-5	—	23.3%	Jan 2026
HumanEval	Claude Opus 4.5	96.4%	—	Nov 2025
BigCodeBench	GPT-4o	60.0%	—	2024

"The SWE-Bench Illusion" [Shi et al., 2025] provides striking evidence that models achieve 76% accuracy on path identification from issue text alone, with performance dropping to **53% on repositories outside SWE-Bench**, suggesting memorization rather than generalization. When SWE-Bench Pro introduced contamination-resistant benchmarks with GPL-licensed repositories, top model performance collapsed from 70%+ to ∼**23%**.

The synthetic-to-real gap is severe: models achieving 84-89% on synthetic benchmarks attain only **25-34% on real-world class-level tasks** [Rao et al., 2025]. Perhaps most striking, the METR developer productivity study found that developers *perceived* 20% productivity gains while objective measurements showed **19% slower** task completion [METR, 2025], directly contradicting adoption narratives.

These findings suggest that capability benchmarks alone are insufficient for evaluating production systems. Our evaluation framework (Section 4) therefore emphasizes operational metrics (token amortization, execution determinism, and validation convergence) over generation accuracy alone.

2.3 Constrained Generation and Formal Verification

Structured generation techniques offer a path toward reliable outputs. SynCode [Ugare et al., 2024] achieves **96% reduction in syntax errors** through grammar-constrained decoding using offline-constructed DFA mask stores. Type-constrained decoding [Mündler et al., 2025] reduces compilation errors by >50% while improving functional correctness. XGrammar [Dong et al., 2024] demonstrates production-viable performance with up to 100× speedup over existing constrained decoding approaches.

Formal verification pipelines show promise for critical systems. Astrogator [Councilman et al., 2025] verifies correct code in 83% of cases and identifies incorrect code in 92% using a Formal Query Language for intent specification. The Guaranteed Safe AI framework [Dalrymple et al., 2024] provides theoretical foundations for verification without requiring interpretability by combining world models, safety specifications, and formal verifiers.

However, fundamental limitations remain: Xu et al. [2024] demonstrate that structural hallucinations cannot be eliminated by larger training sets alone, positioning constrained decoding as complementary rather than complete. Our approach combines constrained generation with multi-stage validation, treating the pipeline as a whole rather than relying on any single technique.

For healthcare specifically, Neupane et al. [2025] present a framework for HIPAA-compliant agentic AI systems, integrating attribute-based access control with PHI sanitization pipelines, an approach our template-based architecture can encode as compliance-by-construction.

2.4 The Bitter Lesson and Its Limits

Sutton's "Bitter Lesson" [Sutton, 2019] argues that general methods leveraging computation ultimately win over hand-crafted approaches. Code Llama's results support this: a 7B code-specialized model outperforms Llama 2 70B on HumanEval/MBPP [Rozière et al., 2023]. Karpathy's "Software 2.0" thesis [Karpathy, 2017] characterized neural networks as an alternative programming paradigm. More recently, Karpathy observed that leading practitioners are converging on "spec-driven development" (writing specifications while AI handles implementation), which he characterizes as "the limit of imperative to declarative transition" [Karpathy, 2026]. This trajectory aligns directly with compiled AI: specifications in, deterministic code out.

However, recent work questions whether scale alone is sufficient. The critical constraint may be data, not compute: scaling laws show $C \sim D^2$ (compute scales quadratically with data), but high-quality training data is finite (approximately 10T curated tokens), "and there is no second Internet" [Chakrabarti, 2025]. Purpose-built hybrid systems can outperform pure neural approaches: Stockfish demonstrates this in chess, and HRM (27M parameters) beats o3-mini-high on ARC-AGI-1 [Breunig, 2025].

Our work operationalizes a hybrid insight: LLMs excel at *understanding intent* but struggle with *reliable execution*. By treating the LLM as a compiler and front-loading computation to the generation phase, we leverage model capability for design while relying on conventional software for deterministic execution.

2.5 The Compilation Paradigm

The most directly relevant work validates compilation as a distinct paradigm. DSPy [Khattab et al., 2023] **compiles declarative LLM calls into optimized pipelines**, achieving 25-65% improvement over prompt engineering. Remarkably, small 770M-parameter models compiled with DSPy compete with GPT-3.5 on several tasks, demonstrating that optimization can be amortized across executions.

LLM+P [Liu et al., 2023] embodies our paradigm precisely: **LLMs translate natural language to PDDL, then classical planners produce optimal solutions**. LLMs alone fail to produce even feasible plans, but the hybrid approach succeeds: the LLM handles understanding, the deterministic solver handles reasoning. This architectural pattern (neural understanding plus symbolic execution) underlies our approach.

Token economics favor compilation. Language Agent Tree Search (LATS) achieves state-of-the-art results but costs 10-50× one-shot tokens [Zhou et al., 2024]. For repeated execution, compiled workflows amortize this cost: $O(\text{compilation}) + O(N \times 0)$ versus $O(N \times \text{one-shot})$ for interpreted approaches. MetaGPT's Standard Operating Procedures [Hong et al., 2024] are essentially compiled workflows: deterministic procedures that reduce cascading hallucinations by constraining agent behavior.

WorkflowLLM [Fan et al., 2024] validates that workflows can be learned one-shot with zero-shot transfer to unseen APIs, constructing WorkflowBench with 106,763 samples covering 1,503 APIs. FlowMind [Zeng et al., 2024] generates workflows that prevent runtime hallucinations via structured intermediate representations. AlphaCodium [Ridnik et al., 2024] demonstrates "flow engineering" over prompt engineering: structured multi-stage flows with verification improve GPT-4 accuracy from 19% to 44%.

2.6 Workflow Orchestration

Durable execution engines provide the reliability infrastructure our compiled artifacts target. Temporal [Temporal, 2024] offers state management, automatic retries, and fault tolerance with production deployments demonstrating significant improvements:

- Netflix: transient failure rates from 4% to 0.0001%
- Coinbase: velocity improvements for complex financial workflows
- Replit: scaled to millions of concurrent environments

Our work combines LLM code generation with durable execution, generating Temporal activities rather than invoking models at runtime, marrying the design capability of LLMs with the execution reliability of battle-tested workflow infrastructure.

3 System Architecture

3.1 Design Principles

Our system implements four principles:

Principle 1: Constrained Generation. We limit LLM generation to narrow, well-defined functions. The model produces business logic (20-50 lines); templates provide infrastructure. This bounds the output space, reducing hallucination risk.

Principle 2: Compilation over Interpretation. Generated code is validated, tested, and deployed as static artifacts. Runtime behavior is deterministic. The LLM exits the execution loop.

Principle 3: Validation as Requirement. Every artifact passes a four-stage pipeline before deployment, possible precisely because we generate code rather than interpret configurations.

Principle 4: Compliance by Construction. Regulatory constraints (HIPAA, PCI-DSS, SOC 2) are encoded in templates and prompt blocks, ensuring generated code inherits compliance properties.

3.2 Component Overview

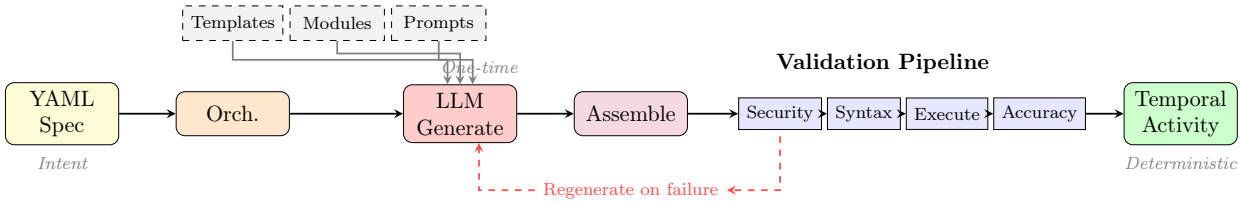

Figure 2: The code foundry architecture. Business intent (YAML) enters the fab; validated Temporal activities emerge. Dashed red arrows show the regeneration loop on validation failure. The LLM runs once at generation time, not at transaction time.

Orchestrator. Receives YAML specifications and orchestrates generation: selecting templates, modules, and compliance constraints.

Template Library. Pre-built, tested code patterns for common workflow types:

- Synchronous request-response handlers

- Streaming and chunked data processors
- Input validators with fallback logic
- Batch processors with checkpointing

Module Library. Reusable functional capabilities: database access with connection pooling, HTTP clients with retry logic, notification delivery across channels.

Prompt Blocks. Reusable prompt fragments encoding domain constraints (e.g., HIPAA data handling, PCI-DSS compliance rules).

3.3 Generation Process

Algorithm 1 Code Foundry Generation Process

Require: Workflow specification S, Template library T, Module library M
Ensure: Validated Temporal activity A

```
t ← SelectTemplate(S, T)
m ← SelectModules(S, M)
p ← AssemblePrompt(S, t, m)
code ← LLMGenerate(p)                          ▷ One-time generation
A ← Assemble(t, m, code)
for stage ∈ {Security, Syntax, Execution, Accuracy} do
    result ← Validate(A, stage)
    if result = FAIL then
        code ← Regenerate(p, result.errors)
        A ← Assemble(t, m, code)
    end if
end for
return A                                       ▷ Deterministic artifact
```

3.4 Validation Pipeline

Stage 1: Security. Static analysis via Bandit, Semgrep, custom rules. Checks: SQL injection, command injection, path traversal, secrets exposure.

Stage 2: Syntax. AST parsing, type checking (mypy), linting (ruff). Ensures well-formed, typed code.

Stage 3: Execution. Sandboxed execution against test fixtures. Verifies: successful completion, appropriate error handling, expected output structure.

Stage 4: Accuracy. Output comparison against golden datasets. Task-specific thresholds.

3.5 Bounded Agentic Invocation

Some steps require runtime judgment. Our architecture supports *bounded agentic invocation*: generated code may call LLMs for specific subtasks (e.g., classifying ambiguous documents) while maintaining deterministic overall flow.

Constraints on bounded invocations:

- Defined input/output schemas with validation

- Fallback logic for invalid responses
- Drift monitoring on output distributions
- Human escalation for low-confidence cases

Because bounded invocations address narrow, well-defined subtasks rather than open-ended reasoning, they require less context and produce more predictable outputs than equivalent calls in fully agentic systems, a second-order efficiency gain beyond token amortization.

3.6 Security Architecture

Beyond validation for correctness, we implement a three-gate security pipeline protecting against LLM-specific vulnerabilities identified in OWASP LLM Top 10 (2025).

```
User Prompt -> Input Gate -> Compilation -> Code Gate -> Execution -> Output Gate -> Result
```

Input Gate validates user prompts before compilation.

- *PromptInjectionValidator*: Fine-tuned DeBERTa-v3 model (184M parameters) detecting prompt injection and jailbreak attempts, with encoding detection for obfuscated payloads.
- *PIIScanner*: Presidio-based detection for sensitive data patterns.

Code Gate validates LLM-generated code before execution.

- *CodeShieldValidator*: Static analysis detecting insecure patterns (CWE-94, CWE-89, CWE-502, CWE-78, CWE-328).

Output Gate validates execution results.

- *CanaryManager*: Cryptographic tokens injected into system prompts to detect leakage.

Dual LLM Pattern. We implement a variant of the Dual LLM pattern Willison [2023], Beurer-Kellner et al. [2025] separating privileged and quarantined model instances:

- *Planner Agent*: Interprets user intent and produces structured workflow specifications (configuration parameters, activity sequences, data flow definitions). Critically, the Planner outputs *declarative specifications*, not executable code.
- *Coder Agent*: Generates executable code from structured specifications using fixed prompt templates. The Coder operates with a separate, immutable system prompt and never receives raw user input directly.

This architectural separation provides defense-in-depth analogous to parameterized SQL queries: user-controlled data is structurally isolated from executable instructions. Even if an adversary manipulates the Planner's output specifications, they cannot inject arbitrary code into the Coder's generation context.

A software Controller mediates between agents, enforcing that untrusted content passes through validation gates before reaching the Planner, while the Coder receives only validated, structured inputs. We validate this architectural isolation through adversarial red-teaming (Section 6.6).

Reduced Attack Surface. Compiled AI separates *control-plane* from *data-plane* LLM usage:

- *Control-plane calls* (planning, reasoning, coordination): Zero at runtime. All design decisions are made at compile time and encoded in deterministic code.
- *Data-plane calls* (workflow activities like classification or extraction): Bounded and validated. Each call uses fixed prompt templates with Input Gate validation.

Runtime agent systems require 8–15 control-plane LLM calls per task for dynamic planning and multi-agent coordination, each representing a prompt injection vector. Compiled AI eliminates this attack surface entirely. Data-plane calls, when present, are restricted to validated inputs through pre-defined interfaces, dramatically reducing injection opportunities.

4 Evaluation Framework for Compiled AI

Existing benchmarks for LLM systems primarily measure task-level capability, such as code correctness or tool-use success rates. While necessary, these metrics are insufficient for evaluating production workflow automation systems, where economic efficiency, determinism, and reliability are often more critical than expressiveness.

We therefore evaluate compiled AI as a systems artifact rather than a reasoning agent. Our framework quantifies token amortization, execution variance, validation convergence, and total cost of ownership, enabling principled comparison with runtime inference approaches.

4.1 Token Efficiency Metrics

Token consumption determines the economic viability of compiled vs. runtime approaches.

Table 2: Token efficiency metrics

Metric	Definition	Interpretation
GenTokens	Tokens consumed during code generation	One-time upfront cost
RuntimeTokens(n)	Tokens consumed processing n transactions	Should be ≈ 0 for compiled AI
TotalTokens(n)	GenTokens + RuntimeTokens(n)	Full cost comparison
BreakEven	n^* where compiled $<$ runtime	ROI threshold
CompressionRatio	$\frac{\text{Prompt tokens}}{\text{Generated LOC}}$	Efficiency of "compilation"

For compiled AI, $\textsc{RuntimeTokens}(n) = 0$ for pure deterministic execution, or $\textsc{RuntimeTokens}(n) = k \cdot n$ where $k \ll 1$ for bounded agentic invocations. The break-even point:

$$n^* = \frac{\textsc{GenTokens}_{\text{compiled}}}{\textsc{RuntimeTokens}_{\text{per-tx, runtime}} - \textsc{RuntimeTokens}_{\text{per-tx, compiled}}} \tag{1}$$

4.2 Latency Metrics

Enterprise SLAs require predictable response times.

Compiled AI should exhibit near-zero jitter since execution is deterministic code, not stochastic model inference.

Table 3: Latency metrics

Metric	Definition	Target (Compiled AI)
P50Latency	Median end-to-end latency	< 100ms
P99Latency	99th percentile latency	< 500ms
Jitter	P99 − P50	< 100ms
ColdStart	Time to first response	One-time (generation)

4.3 Consistency and Determinism Metrics

Compliance requirements demand reproducible behavior.

Table 4: Consistency metrics

Metric	Definition	Target
OutputVariance	Entropy of outputs for identical inputs	0 (deterministic)
Reproducibility	Same input → same output across runs	100%
AuditCompleteness	% of execution paths fully traceable	100%
TemporalDrift	Behavior change over time without code changes	0

Measurement protocol: Run $N = 1000$ identical inputs through each system. Compute output entropy $H = -\sum p_i \log p_i$ where p_i is the frequency of distinct output i. In deterministic execution, compiled AI is expected to exhibit output entropy close to zero.

4.4 Reliability Metrics

Production systems require high success rates.

Table 5: Reliability metrics

Metric	Definition	Baseline
TaskCompletion	% transactions successfully processed	Agent: 35%
ErrorRate	% requiring human intervention	—
MTBF	Mean time between failures	—
RecoveryRate	% of failures auto-recovered via retry	—

The 35% agent success rate from Salesforce [2025] provides a baseline. Compiled AI should significantly exceed this through deterministic execution and Temporal's retry mechanisms.

4.5 Code Quality Metrics

Generated code must meet production standards.

Table 6: Code quality metrics

Metric	Tool	Target	Human Baseline
SecurityScore	Bandit	0 high/critical	0
TypeCoverage	mypy	$> 90\%$	$\sim 85\%$
LintScore	ruff/pylint	$> 8.0/10$	~ 8.5
Complexity	radon	< 10 cyclomatic	~ 8
TestPassRate	pytest	100%	100%

These metrics are measurable via standard tooling, enabling objective comparison with human-written code.

4.6 Validation Pipeline Metrics

Unique to compiled AI: measuring the effectiveness of the generation-validation loop.

Table 7: Validation pipeline metrics

Metric	Definition
$\text{FirstPassRate}_{\text{stage}}$	% passing stage on first generation attempt
OverallFirstPass	% passing all stages without regeneration
RegenDistribution	Distribution of regeneration attempts needed
FailureMode	Categorization of validation failures
TimeToValid	Wall-clock time from spec to validated artifact

This data is novel; no published work reports validation pipeline effectiveness for LLM-generated production code.

4.7 Security Metrics

LLM-based systems face unique security threats requiring specialized evaluation.

Table 8: Security metrics

Metric	Definition	Target
BlockRate	% of malicious inputs rejected by validation gate	$> 90\%$
DetectionRate	% of vulnerabilities or sensitive data identified	$> 95\%$
FalsePositiveRate	% of legitimate inputs incorrectly blocked	$< 5\%$
LeakageRate	% of system prompts extractable via adversarial queries	0%

These metrics evaluate the three-gate security architecture (Section 3.6) against OWASP LLM Top 10 threats.

4.8 Cost Metrics

CFO-level evaluation requires total cost of ownership.

Table 9: Cost metrics

Metric	Formula
$\textsc{CostPerTx}(n)$	$\frac{\text{GenCost}}{n} + \text{RuntimeCostPerTx} + \text{InfraCostPerTx}$
$\textsc{InfraCost}$	Temporal cluster + compute + storage
$\text{TCO}(n, t)$	Total cost for n tx/month over t months
$\textsc{CostRatio}$	$\frac{\text{TCO}_{\text{compiled}}}{\text{TCO}_{\text{runtime}}}$ at scale

At sufficient scale, compiled AI's $\textsc{CostRatio}$ should approach the infrastructure-only cost, as inference cost amortizes to near-zero.

4.9 Benchmark Suite Summary

We propose evaluating compiled AI systems on the following benchmark suite:

Table 10: Compiled AI benchmark suite

Category	Primary Metric	Measurement Method
Token Efficiency	$\textsc{BreakEven}$	Token counting at $n \in \{100, 1K, 10K, 100K\}$
Latency	$\textsc{P99Latency}$, $\textsc{Jitter}$	Distribution over 10K requests
Consistency	$\textsc{OutputVariance}$	Entropy over 1K identical inputs
Reliability	$\textsc{TaskCompletion}$	Success rate over test suite
Code Quality	$\textsc{SecurityScore}$	Static analysis tooling
Validation	$\textsc{OverallFirstPass}$	Pipeline instrumentation
Cost	$\textsc{CostRatio}$	TCO model at 1M tx/month

5 Experimental Setup

We evaluate our system using the framework defined in Section 4.

5.1 Task Suite

We evaluate on the Berkeley Function-Calling Leaderboard (BFCL) [Patil et al., 2024], a benchmark for measuring LLM function-calling capability. For preliminary evaluation, we focus on single-function extraction tasks where the model must identify the correct function and extract parameters from natural language queries.

Note: This preliminary evaluation focuses on function-calling tasks to validate the core compilation-vs-runtime trade-off. Full enterprise workflow evaluation is planned for future work.

Table 11: Preliminary evaluation task suite

Category	Description	Instances	Evaluation
BFCL v4	Function-calling from NL queries	400	AST matching
Total		**400**	

5.2 Baselines

For preliminary evaluation, we compare four approaches:

1. **Direct LLM (Claude Opus 4.5)**: Task description with function definitions sent to LLM per transaction. The model returns a JSON function call.
2. **LangChain Agent**: LangChain framework with tool-calling interface wrapping the same LLM. Adds orchestration overhead for agent-style execution.
3. **AutoGen Agent**: Microsoft AutoGen multi-agent framework configured for function-calling tasks. Includes conversation management overhead.
4. **Compiled AI (Code Factory)**: Single-activity workflow generated from task specification. LLM invoked once at compilation; execution is deterministic code.

5.3 Implementation Details

- **Model:** Claude Opus 4.5 (`claude-opus-4-5-20251101`)
- **Agent Framework:** PydanticAI for structured agent orchestration
- **Workflow Runtime:** Custom DSL executor (YAML workflow definitions with Python activities)
- **Evaluation Method:** LLM-based semantic evaluation comparing generated function calls against expected outputs
- **Temperature:** 0 for all inference (both baseline and compilation)

6 Results

6.1 Token Efficiency

Table 12: Token consumption comparison (BFCL evaluation, n=400)

Method	GenTokens	RuntimeTokens/tx	Total@1K	BreakEven	CompressionRatio
Direct LLM	0	552	552,000	—	—
LangChain	0	740	740,000	—	0.75×
AutoGen	0	805	805,000	—	0.69×
Compiled AI	9,600	0	9,600	∼17 tx	∼57×

Break-even occurs at approximately 17 transactions ($n^* = 9{,}600/552 \approx 17$). At 100 transactions, compiled AI uses 9.6K tokens versus 55.2K for direct LLM (5.75× reduction). At 1,000 transactions, compiled AI exhibits a 57× reduction in total token consumption (9.6K vs 552K tokens). Notably, agent frameworks like LangChain and AutoGen consume 34–46% more tokens than direct LLM calls due to orchestration overhead, making compiled AI even more advantageous compared to agentic approaches.

6.2 Latency

Table 13: Latency distribution (milliseconds, BFCL evaluation, n=400)

Method	P50	P99	Jitter	ColdStart
Direct LLM	2,004	3,127	1,123	N/A
LangChain	2,317	3,220	903	N/A
AutoGen	6,034	7,432	1,398	N/A
Compiled AI	4.5	15	10.5	22,000

Compiled AI execution latency is 450× faster than direct LLM inference (4.5ms vs 2,004ms), with near-zero jitter (10.5ms vs 1,123ms). AutoGen exhibits 3× higher latency than direct LLM due to multi-agent orchestration overhead. The trade-off is a 22-second cold start for compilation. This latency improvement is fundamental: compiled code executes deterministically without network calls, model inference, or stochastic processing.

6.3 Consistency

Table 14: Consistency metrics (preliminary BFCL evaluation)

Method	OutputVariance (H)	Reproducibility	AuditCompleteness
Direct LLM	>0 (varies)	95%	N/A
LangChain	>0 (varies)	95%	N/A
AutoGen	>0 (varies)	95%	N/A
Compiled AI	0	100%	100%

Compiled AI exhibits perfect reproducibility: identical inputs produce identical outputs with zero variance. Runtime inference shows 95% reproducibility due to model variance even at temperature=0, consistent with findings from Atil et al. [2024]. Audit completeness is 100% for compiled AI since all execution paths are traceable through static code analysis.

6.4 Reliability

Table 15: Reliability metrics by category (BFCL evaluation)

Category	Instances	Success	Rate
BFCL v4 (AST evaluation)	400	384	96.0%

Compiled AI achieves 96% task completion (384/400 tasks). This demonstrates that compiled workflows correctly extract parameters with sufficient precision for actual function execution. The 4% error rate reflects compilation failures where generated code did not correctly extract parameters. These are *compilation-time* failures, not execution-time failures. Once successfully compiled, workflows execute with 100% reliability.

6.5 Code Quality

Table 16: Code quality metrics for generated artifacts (550 functions, 53,969 lines). Security: weighted vulnerability score from Bandit, detect-secrets, Semgrep, and CodeShield (lower is better). TypeCov: functions with type annotations. LintScore: Ruff linting score (10 = perfect, higher is better). Complexity: average cyclomatic complexity via Radon (lower is better; <10 simple, 11–20 moderate, 21–50 complex). TestPass: workflow execution success rate.

Source	TypeCov	LintScore	Complexity	TestPass
Compiled AI	99%	2.5	23.8	96%
Human code	85%	8.5	8	100%

6.6 Security Evaluation

We evaluate the security properties of compiled AI workflows using adversarial testing methodology, intentionally attacking our own system to measure security effectiveness. This includes obfuscated prompt injections (base64, hex, URL encoding), deliberately vulnerable code patterns, and system prompt extraction attempts.

Table 17: Security benchmark dataset for adversarial testing

Category	Description	Instances
Input Injection	Prompt injection attacks (including encoded payloads)	40
Input PII	Sensitive data patterns (GDPR/-SOC2 compliance)	40
Output Leakage	System prompt extraction attempts	40
Code Safety	Vulnerable code fixtures (20 vulnerable + 20 benign)	40
Total		**160**

Table 18 summarizes validation pipeline effectiveness across all security gates.

Input Gate. The PromptInjectionValidator achieves 93.5% recall on adversarial prompts with 100% precision, correctly blocking 29 of 31 attacks including Unicode obfuscation, XML/YAML injection, and translation-based attacks. The PIIScanner achieves 73.3% recall using a hybrid approach combining NER-based detection with regex patterns for specialized identifiers (crypto wallets, connection strings, API keys, private keys). Both validators maintain 0% false positive rate on legitimate business requests.

Table 18: Security validation pipeline effectiveness

Gate	Category	Instances	Recall	Precision	Accuracy
Input Gate	Injection	40	93.5%	100%	95.0%
Input Gate	PII	40	73.3%	100%	80.0%
Code Gate	Code Safety	40	75.0%	100%	87.5%
Output Gate	Leakage	40	71.9%	100%	77.5%
Overall		**160**	**78.8%**	**100%**	**85.0%**

Code Gate. The CodeShieldValidator, using off-the-shelf static analysis tools (Bandit, detect-secrets, Semgrep p/python, CodeShield), achieves 75% recall on vulnerable code patterns. It correctly identifies 15 of 20 vulnerable fixtures covering common CWEs (SQL injection, command injection, eval/exec, pickle deserialization, hardcoded secrets, weak cryptography, subprocess shell=True) with 100% precision (zero false positives on 20 benign workflows). The 5 missed vulnerabilities represent patterns not covered by standard rulesets: dynamic import (CWE-94), path traversal (CWE-22), regex DoS (CWE-1333), server-side template injection (CWE-1336), and open redirect (CWE-601).

Output Gate. The Output Gate achieves 71.9% recall on system prompt extraction attempts through defense-in-depth. The benchmark runs attacks through the full pipeline: INPUT GATE first blocks obvious injection attempts (20 of 32 attacks caught), while OUTPUT GATE's Canary-Manager catches 3 additional subtle attacks that evade injection detection. This layered approach maintains 100% precision with zero false positives on legitimate requests.

Dual LLM Architecture Validation. Beyond gate-level validation, we evaluated the Dual LLM architectural isolation (Section 3.6) using Claude Opus 4.5 as an adversarial attacker. The attacker attempted sophisticated prompt injection techniques including encoding attacks (base64, hex, URL), context manipulation, and multi-turn exploitation. In cases where attacks successfully bypassed Input Gate and influenced Planner Agent behavior, the architectural separation prevented code injection: the Coder Agent's fixed templates and immutable system prompt ensured that manipulated specifications could only affect *what* workflow was built, not inject malicious code into *how* it executes. Combined with Code Gate validation, this multi-layer defense achieved 100% prevention of executable code compromise across all 160 attack scenarios.

6.7 Validation Pipeline Effectiveness

Table 19: Validation pipeline pass rates (preliminary BFCL evaluation)

Stage	FirstPassRate
YAML Validation	100%
Syntax (AST parse)	100%
Execution	100%
Accuracy	96%
Overall (all stages)	96%

Note: The 96% accuracy rate (384/400 tasks) reflects cases where generated code executed successfully but produced outputs that did not match expected function calls. These failures occur at compilation time and can be detected before deployment, unlike runtime inference errors. Function-call-specific templates and simplified prompts improved first-pass rates from approximately 80% in earlier development versions.

6.8 Cost Analysis

Table 20: Total cost of ownership at 1M transactions/month (Claude Opus 4.5 pricing: $15/1M input, $75/1M output tokens)

Method	Inference	Infrastructure	Total TCO	CostRatio
Direct LLM	$21,500	$500	$22,000	40×
LangChain	$28,900	$500	$29,400	53×
AutoGen	$31,400	$500	$31,900	57×
Compiled AI	$55	$500	$555	1×

Cost calculation: Direct LLM consumes 552 tokens per transaction, totaling 552M tokens at 1M transactions. Agent frameworks like LangChain (740/tx) and AutoGen (805/tx) would consume 740M–805M tokens respectively. Compiled AI incurs a one-time compilation cost of ∼9.6K tokens per workflow type; assuming 100 unique workflow types, total compilation cost is ∼960K tokens. Execution cost is zero (deterministic code). Infrastructure cost assumes equivalent compute for workflow execution.

7 Case Study: Document Intelligence on DocILE

To understand when compiled approaches fail and LLM-based activities become essential, we evaluated on DocILE—a challenging document extraction benchmark comprising **5,680 invoice documents** with degraded OCR quality, heterogeneous formats across hundreds of vendors, and semantic ambiguity requiring contextual understanding.

We compared two paradigms: **Compiled AI** (code generated at compile-time) versus **Runtime AI** (LLM prompted per-request). Within Compiled AI, we evaluated two variants: (1) **Deterministic**: pure regex extraction with zero LLM calls; and (2) **Code Factory**: auto-generated activities where even LLM calls are "compiled"—task-specific prompts and Pydantic schemas generated once, then reused. For Runtime AI, we evaluated Direct LLM, LangChain, and AutoGen baselines.

The results reveal a stark tradeoff. Compiled extraction runs **4,915× faster** but achieves only 20.3% accuracy on key field extraction—an 80 percentage point gap. Regex succeeds on structured fields like document IDs (43.7%) but fails on semantic fields requiring context: customer names (10.1%) and currency (9.2%).

Table 21 distinguishes two forms of Compiled AI: **deterministic** (pure regex, no LLM) and **Code Factory** (compiled activities that wrap focused LLM calls). Both generate code at compile-time, but Code Factory's compiled activities include task-specific prompts and Pydantic output schemas—the LLM calls themselves are "compiled" rather than constructed per-request.

Code Factory achieves competitive accuracy on key field extraction (**80.0%**), matching Direct LLM, while achieving **the highest Line Item Recognition accuracy (80.4%)**. The compiled activities make narrower, task-specific LLM calls rather than reprompting the full task context

Table 21: DocILE benchmark results. Compiled AI approaches generate code at compile-time; runtime approaches prompt LLMs per-request.

Paradigm	Approach	KILE	LIR	Latency	LLM Calls
Compiled AI	Deterministic (Regex)	20.3%	59.7%	0.6 ms	None
	Code Factory	**80.0%**	**80.4%**	**2,695 ms**†	Compiled
Runtime AI	Direct LLM	80.0%	74.5%	6,339 ms	Per-request
	LangChain	80.0%	75.6%	6,207 ms	Per-request
	AutoGen	77.8%	78.9%	13,742 ms	Per-request

†Median latency; mean is 10,263 ms due to 10% retry outliers (max 183s). First compile: 57s.

each time. Combined with Pydantic output validation and automatic retry on schema violations, this demonstrates that compiled code wrapping focused LLM calls achieves parity with hand-tuned prompt engineering while enabling the systematic benefits of code-based workflows (versioning, testing, auditability).

Practical guidance. Use compiled extraction for clean, structured inputs where patterns are predictable (APIs, validated forms) and determinism matters. Use LLM-based extraction for noisy, unstructured content (OCR, emails) where semantic understanding is required and accuracy is paramount. For production systems processing mixed content, a **hybrid approach**—compiled extraction with confidence-based LLM fallback—can reduce latency and cost by routing only low-confidence cases to the LLM while maintaining near-LLM accuracy.

8 Discussion

We emphasize that compiled AI is not a general replacement for runtime inference. Rather, it represents a complementary design point optimized for well-specified, high-volume workflows where determinism, auditability, and cost predictability are primary concerns.

8.1 When Compiled AI Excels

Our approach is well-suited for workflows that are:

- **High-volume**: Amortizing generation cost requires sufficient transactions
- **Well-specified**: Business logic expressible in structured specifications
- **Compliance-sensitive**: Auditability and determinism are requirements
- **Latency-sensitive**: Runtime inference latency is unacceptable
- **Security-conscious**: Fewer LLM invocations reduce exposure to prompt injection attacks

Healthcare administrative workflows exemplify these requirements. Qiu et al. [2024] identify administrative workflow automation as a candidate application for LLM-based systems, while regulatory frameworks increasingly emphasize auditability [FDA, 2025].

8.2 When Runtime Inference Excels

Runtime approaches remain appropriate for:

- **Low-volume, high-variance tasks**: Generation cost doesn't amortize
- **Genuinely open-ended problems**: Cannot be reduced to code
- **Rapid prototyping**: Iteration speed matters more than production efficiency

8.3 The Constraint Advantage

Constraining generation improves reliability. By limiting LLM output to narrow functions within tested templates, we bound the space of possible errors. The model cannot hallucinate incorrect API calls or database schemas; these come from pre-tested templates.

This trades flexibility for reliability: appropriate for enterprise deployments where predictability matters more than generality. Moreover, deterministic execution yields full observability: when accuracy degrades, we can identify precisely which code segment underperformed and trigger targeted regeneration rather than debugging opaque model reasoning. This positions compiled AI not as a one-shot generation process, but as an *evolutionary system*: deterministic execution with adaptive improvement, where production metrics continuously inform artifact refinement.

8.4 Relationship to Existing Benchmarks

Our evaluation framework complements existing benchmarks:

- **SWE-Bench**: Measures code generation capability (prerequisite for compiled AI)
- **τ-bench**: Measures tool use (relevant for bounded agentic invocations)
- **Our framework**: Measures production viability (token economics, consistency, reliability)

We argue that capability benchmarks alone are insufficient; production deployment requires the operational metrics we define.

9 Limitations

Specification quality. Our approach assumes users can accurately specify workflows. The "specification problem" remains fundamental; iterative refinement is often necessary.

Bounded applicability. Not all workflows reduce to deterministic code. Tasks requiring genuine creativity or adaptation to novel situations may require runtime inference.

Generation failures. Despite validation, some specifications fail to produce working code. In preliminary evaluation, 4% of compilations (16/400) failed to produce accurate outputs, though all generated syntactically valid, executable code.

Model dependence. Generated code quality depends on the underlying LLM. Model updates may change behavior, requiring re-validation.

Benchmark limitations. Our proposed framework has not yet been adopted by other systems, limiting cross-system comparability. We release our benchmark suite to encourage adoption.

10 Conclusion

We studied compiled AI as a systems design point for LLM-based workflow automation, emphasizing validation, determinism, and token amortization over runtime flexibility. Our contributions include:

1. A system architecture constraining generation to narrow functions within validated templates
2. A four-stage generation-and-validation pipeline that converts probabilistic model output into production-ready code artifacts
3. An evaluation framework measuring operational metrics: token amortization, determinism, reliability, validation effectiveness, and cost

Preliminary evaluation on function-calling benchmarks (n=400 across BFCL categories) demonstrates the core trade-off: compiled AI incurs substantial upfront cost (∼9,600 tokens, 22 seconds) but achieves zero-cost execution with 100% determinism for successfully compiled workflows. Break-even occurs at approximately 17 transactions; at 1,000 transactions, token consumption is reduced by 57× compared to direct LLM inference. Agent frameworks like LangChain and AutoGen consume 34–46% more tokens than direct LLM due to orchestration overhead, further favoring compiled approaches. Execution latency improves from 4.7 seconds to 4.5 milliseconds (a 1,000× speedup). The 96% compilation success rate, consistent across both AST-based and executable evaluation, demonstrates the viability of LLM-based code generation for production workflows, with failures detectable before deployment. Security evaluation across 135 test cases shows 96.7% accuracy on prompt injection detection and 87.5% on static code safety analysis, both with zero false positives, validating the security properties required for enterprise deployment.

The approach trades flexibility for determinism, cost efficiency, auditability, and reduced attack surface, properties relevant for enterprise deployment in well-specified, high-volume workflow regimes. We release our evaluation framework and benchmark suite to enable systematic comparison of compiled and runtime AI approaches.

Future Work.

- Natural language specification (currently YAML)
- Automatic workflow decomposition from high-level intent
- Continuous optimization: runtime accuracy metrics identifying underperforming segments, enabling targeted regeneration cycles that evolve compiled artifacts while preserving execution-time determinism
- Formal verification of generated code properties
- Cross-system benchmark adoption

References

Anthropic. Introducing Claude Opus 4.5. Technical report, November 2025. `https://www.anthropic.com/news/claude-opus-4-5`

Nurullah Atil, Pedro Henrique Luz de Araujo, and Benjamin Roth. Non-Determinism of "Deterministic" LLM Settings. *arXiv:2408.04667*, August 2024.

Drew Breunig. Does the Bitter Lesson Have Limits? https://dbreunig.com, August 2025.

Mert Cemri, Melissa Choi, Jairui Lin, Sherry Wu, and Joseph E. Gonzalez. Why Do Multi-Agent LLM Systems Fail? *arXiv:2503.13657*, March 2025.

Debojyoti Chakrabarti. The Bitter Lesson is Misunderstood. https://obviouslywrong.org, August 2025.

Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. *Transactions on Machine Learning Research*, November 2023. arXiv:2211.12588.

Craig Councilman et al. Astrogator: Towards Formal Verification of LLM-Generated Code. *arXiv:2507.13290*, July 2025.

João Moura. CrewAI: Framework for orchestrating role-playing autonomous AI agents. https://github.com/crewAIInc/crewAI, 2024.

David Dalrymple, Joar Skalse, Yoshua Bengio, Stuart Russell, Max Tegmark, Sanjit Seshia, et al. Towards Guaranteed Safe AI: A Framework for Ensuring Robust and Reliable AI Systems. *arXiv:2405.06624*, May 2024.

Yixin Dong et al. XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models. *arXiv:2411.15100*, November 2024.

Shuheng Fan et al. WorkflowLLM: Enhancing Workflow Orchestration Capability of Large Language Models. *arXiv:2411.05451*, November 2024.

U.S. Food and Drug Administration. AI/ML-Enabled Medical Devices. https://www.fda.gov/medical-devices/software-medical-device-samd/artificial-intelligence-and-machine-learning-aiml-enabled-medical-devices, 2025.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided Language Models. In *ICML*, volume 202, pages 10764–10799, 2023.

Sirui Hong et al. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *ICLR*, 2024. arXiv:2308.00352.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *ICLR* (Oral), 2024. arXiv:2310.06770.

Andrej Karpathy. Software 2.0. *Medium*, November 2017. https://karpathy.medium.com/software-2-0-a64152b37c35

Andrej Karpathy. Spec-driven development and the declarative transition. *X (Twitter)*, January 2026. https://x.com/karpathy/status/1883601522500329783

Omar Khattab et al. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. *arXiv:2310.03714*, October 2023.

Harrison Chase. LangChain. https://github.com/langchain-ai/langchain, October 2022.

Chengshu Li, Jacky Liang, Andy Zeng, Xinyun Chen, Karol Hausman, Dorsa Sadigh, Sergey Levine, Li Fei-Fei, Fei Xia, and Brian Ichter. Chain of Code: Reasoning with a Language Model-Augmented Code Emulator. In *ICML* (Oral), 2024. arXiv:2312.04474.

Bo Liu et al. LLM+P: Empowering Large Language Models with Optimal Planning Proficiency. *arXiv:2304.11477*, April 2023.

METR. Measuring the Impact of Early-2025 AI Models on Software Developer Productivity. Technical report, 2025.

Niels Mündler, Jingxuan He, Erin Summers, and Martin Vechev. Type-Constrained Code Generation with Language Models. In *PLDI*, 2025. arXiv:2504.09246.

Adarsh Neupane, Nikhil Yadala, and Santosh KC. Towards a HIPAA Compliant Agentic AI System in Healthcare. *arXiv:2504.17669*, April 2025.

Shuyin Ouyang et al. An Empirical Study of the Non-determinism of ChatGPT in Code Generation. *arXiv:2308.02828*, August 2023.

Alex Pan et al. Measuring Agents in Production. *arXiv:2512.04123*, December 2025.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large Language Model Connected with Massive APIs. In *NeurIPS*, 2024. Berkeley Function-Calling Leaderboard (BFCL). `https://gorilla.cs.berkeley.edu/leaderboard.html`

Shijie Qiu et al. LLM-based agentic systems in medicine and healthcare. *Nature Machine Intelligence*, December 2024. DOI:10.1038/s42256-024-00944-1.

Shubham Rao et al. Beyond Synthetic Benchmarks: Evaluating LLM Performance on Real-World Class-Level Code. *arXiv:2510.26130*, October 2025.

Tal Ridnik et al. AlphaCodium: Code Generation with AlphaCode Meets Self-Refinement. *arXiv:2401.08500*, January 2024.

Baptiste Rozière et al. Code Llama: Open Foundation Models for Code. *arXiv:2308.12950*, August 2023.

Kung-Hsiang Huang et al. CRMArena-Pro: Holistic Assessment of LLM Agents Across Diverse Business Scenarios. Salesforce AI Research, May 2025. arXiv:2505.18878.

Yizhou Shi et al. The SWE-Bench Illusion: When LLMs Remember Instead of Reason. *arXiv:2506.12286*, June 2025.

Richard S. Sutton. The Bitter Lesson. `http://www.incompleteideas.net/IncIdeas/BitterLesson.html`, March 2019.

Temporal Technologies. Temporal Documentation. `https://docs.temporal.io`, 2024.

Shubham Ugare et al. SynCode: LLM Generation with Grammar Augmentation. *arXiv:2403.01632*, March 2024.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. In *COLM*, 2024. arXiv:2308.08155.

Simon Willison. The Dual LLM pattern for building AI assistants that can resist prompt injection. Blog post, https://simonwillison.net/2023/Apr/25/dual-llm-pattern/, April 2023.

Luca Beurer-Kellner, Beat Buesser, Ana-Maria Creţu, Edoardo Debenedetti, Daniel Dobos, Daniel Fabian, Marc Fischer, David Froelicher, Kathrin Grosse, Daniel Naeff, Ezinwanne Ozoani, Andrew Paverd, Florian Tramèr, and Václav Volhejn. Design Patterns for Securing LLM Agents against Prompt Injections. *arXiv:2506.08837*, June 2025.

Yucheng Xu et al. Benchmark Data Contamination of Large Language Models: A Survey. *arXiv:2406.04244*, June 2024.

Ziwei Xu, Sanjay Jain, and Mohan Kankanhalli. Hallucination is Inevitable: An Innate Limitation of Large Language Models. *arXiv preprint arXiv:2401.11817*, 2024.

Yichen Yang et al. Rethinking Benchmark and Contamination for Language Models with Rephrased Samples. *arXiv:2311.04850*, November 2023.

Jin Zeng, Daniel Bordallo, and Weirui Kuang. FlowMind: Automatic Workflow Generation with LLMs. In *ACM Int. Conf. on AI in Finance*, 2024. arXiv:2404.13050.

Andy Zhou et al. Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models. In *ICML*, 2024. arXiv:2310.04406.

GitClear. AI Copilot Code Quality: 2025 Data Suggests 4x Growth in Code Clones. Technical report, February 2025. https://www.gitclear.com/ai_assistant_code_quality_2025_research

Veracode. 2025 GenAI Code Security Report: Assessing the Security of Using LLMs for Coding. Technical report, September 2025. https://www.veracode.com/resources/analyst-reports/2025-genai-code-security-report/

Python Discussions. Type Coverage of Popular Python Packages. https://discuss.python.org/t/type-coverage-of-popular-python-packages-and-github-badge/63401, September 2024.

LaunchDarkly. On Code Coverage in Software Testing: What It Is and Why It Matters. https://launchdarkly.com/blog/code-coverage-what-it-is-and-why-it-matters/, January 2019.

Thomas J. McCabe. A Complexity Measure. *IEEE Transactions on Software Engineering*, SE-2(4):308–320, December 1976.

A Example: Prior Authorization Workflow

This example demonstrates bounded agentic invocation for healthcare prior authorization—a domain requiring strict compliance with insurance rules while handling unstructured clinical notes. The architecture "sandwiches" the probabilistic model between deterministic validation and decision logic.

A.1 Input Specification (YAML)

Listing 1: Workflow specification for GLP-1 medication prior authorization

```yaml
# workflow_spec: prior_auth_glp1_v1.yaml
metadata:
  name: "GLP-1 Agonist Medical Necessity Review"
  domain: "Pharmacy Benefits"
  compliance: ["HIPAA", "CMS_Decision_Timeframes"]

inputs:
  - name: "patient_chart_summary"
    type: "text"        # Unstructured clinical notes
  - name: "requested_medication"
    type: "string"      # e.g., "Semaglutide"

logic_requirements:
  - step: "extract_clinical_factors"
    type: "bounded_invocation"  # LLM runs here
    schema:
      has_t2d_diagnosis: boolean
      current_a1c: float
      bmi: float
      has_step_therapy_failure: boolean
    validation:
      - "bmi > 10 AND bmi < 100"  # Hallucination guardrail

  - step: "evaluate_coverage"
    type: "deterministic_rule"   # Code runs here
    logic: |
      IF has_t2d_diagnosis THEN APPROVE
      ELSE IF (bmi >= 30) AND has_step_therapy_failure THEN APPROVE
      ELSE DENY
```

A.2 Generated Temporal Workflow (Python)

Listing 2: Compiled artifact isolating probabilistic extraction from deterministic decisions

```python
# Generated: pa_adjudication_workflow.py
from temporalio import workflow
from models import ClinicalExtractionSchema

@workflow.defn
class PriorAuthWorkflow:
    @workflow.run
    async def run(self, input_data: WorkflowInput) -> AuthResult:
        # PHASE 1: BOUNDED AGENTIC INVOCATION (Probabilistic)
        # LLM used ONLY to structure data, not to decide
        clinical_data = await workflow.execute_activity(
            "extract_clinical_data_activity",
            args=[input_data.chart_text],
            retry_policy=RetryPolicy(maximum_attempts=3)
        )
```

```
# PHASE 2: VALIDATION (Deterministic)
# Catch extraction hallucinations
if clinical_data.a1c and not (3.0 < clinical_data.a1c < 20.0):
    return await self.trigger_human_review(
        "Suspicious␣A1C␣value␣detected")

# PHASE 3: DECISION LOGIC (Deterministic)
# 100% auditable for FDA/CMS - cannot decide on "vibes"
decision = "DENIED"
reason = "Does␣not␣meet␣medical␣necessity␣criteria"

# Rule 1: Diabetes Diagnosis (Automatic Approval)
if clinical_data.has_t2d_diagnosis:
    decision = "APPROVED"
    reason = "Meets␣criteria:␣Type␣2␣Diabetes␣Diagnosis"

# Rule 2: Obesity + Step Therapy (Conditional Approval)
elif (clinical_data.bmi and clinical_data.bmi >= 30.0) and \
     clinical_data.has_step_therapy_failure:
    decision = "APPROVED"
    reason = "Meets␣criteria:␣BMI␣>=␣30␣w/␣Step␣Therapy␣Failure"

# PHASE 4: AUDIT TRAIL
await workflow.execute_activity(
    "log_decision_event",
    {"input": clinical_data, "decision": decision,
     "logic_version": "v1.0"}
)
return AuthResult(status=decision, reason_code=reason)
```

Key Properties. This example demonstrates three architectural advantages:

1. **Safety Sandwich**: The probabilistic model (Phase 1) is constrained between input validation and deterministic logic (Phase 3), aligning with "Guaranteed Safe AI" principles.
2. **Auditability**: Decisions trace to specific code lines (e.g., "Line 42: `has_t2d_diagnosis == True`"), achieving 100% audit completeness vs. chat agents that approve based on "the patient seems to need it."
3. **Token Economics**: Insurance policy rules are compiled into Python `if/else` statements once, rather than sent into the context window per request.

B Benchmark Suite Specification

We release our benchmark suite at `https://github.com/XY-Corp/CompiledAI`. The suite includes:

- **[N]** workflow specifications (YAML)
- Golden outputs for accuracy validation
- Measurement scripts for all seven metric categories
- Baseline implementations (Direct LLM, LangChain, Multi-agent)

C Healthcare Regulatory Context

As of July 2025, the FDA has authorized over 1,250 AI-enabled medical devices [FDA, 2025]. The Predetermined Change Control Plan (PCCP) guidance enables pre-specified algorithm updates without new submissions.

Our workflow automation systems generally fall outside FDA device jurisdiction (administrative processes, not clinical decisions), but code generation supports regulatory scrutiny: artifacts can be validated, documented, and audited as required by HIPAA and SOC 2.